



Cake List App

Overview

Starting from scratch, develop a simple mobile app that loads and displays a list of cakes, from the data source given below.

You may complete this exercise using whatever mobile technology you choose (e.g. iOS/Swift, Android/Kotlin, React Native, Flutter).

Our stance on AI

You may utilise AI assistants in creating your response, but we would expect this to be disclosed to our marking team at time of submission or in a readme file. If AI enables you to build higher quality solutions faster, then we're keen to see that, but a simple fully AI submission generally doesn't score well in our marking criteria.

This exercise is an opportunity to demonstrate your mobile development capabilities, but **we are not expecting perfection**. We've planned this exercise to take no more than a few hours, so focus on the fundamentals, not on addressing all unhappy paths, visual polish, etc.

Data Source

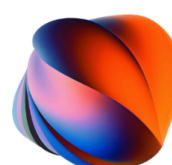
The cakes list should be loaded by treating the following raw file as an API endpoint. Your app will make an HTTP request to this URL to fetch the data:

<https://raw.githubusercontent.com/Waracle/mobile-coding-test-api/refs/heads/main/cakes>

Functionality

Must have

1. When the app is started, load and display a list of cakes
 - a. Remove duplicate entries
 - b. Order entries by *name*
 - c. Display *image* and *title* for each entry
 - d. Display a divider between each entry
2. Display the cake *description* in some kind of popup when a cake entry is clicked





3. Provide some kind of refresh option that reloads the list
4. Display an error message if the list cannot be loaded (e.g. no network)

Nice to have

1. Handle orientation changes, ideally without reloading the list
2. Provide an option to *retry* when an error is presented
3. Animate in list items (e.g. *fade in* or *fall down* animations)

Technical Requirements

1. **Develop using your mobile technology of choice (e.g. iOS/Swift, Android/Kotlin, React Native, Flutter) or as requested by Waracle**
2. Use common 3rd party libraries where appropriate and required
3. Apply a common architectural pattern idiomatic to your chosen platform (e.g. MVVM, MVI, MVP, Clean Architecture, BLoC)
4. Use `TODOs` to document additions you would provide if more time
5. Use Git commits to demonstrate how you tackled the exercise

Unit Tests

Prove the value of the selected architectural pattern by providing unit tests for at least the following business logic (tests should be fast, isolated unit tests using mocks / fakes / stubs for components not under test, rather than UI or end-to-end integration tests)

1. When app is started and load succeeds, cakes are presented with duplicates removed and sorting applied
2. When app is started and load fails, error is presented

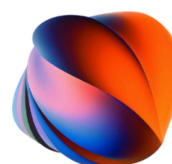
Deliverables

Git repository

Please provide your submission in the form of a publicly accessible Git repository.

Runnable Project & Setup Instructions

The repository should contain a standard project for your chosen technology (e.g.





Xcode project/workspace for iOS, Gradle project for Android, or standard React Native / Flutter project) that can be opened and run on a simulator, emulator, or physical device.

Assessment Criteria

Things that are important to us include

- Code style and cleanliness
- Lightweight application of common libraries and techniques
- Idiomatic application of platform UI components (minimal visual styling required for this exercise)
- Clear code architecture that lends itself to testability

Things that are **less** important to us for this exercise

- Addressing every scenario / unhappy path / edge case (document with `TODOs` so we know you are aware of what has **not** been built)
- Demonstrating knowledge of every common Android library / framework (just use what adds genuine value)
- 100% test coverage
- Visual perfection

